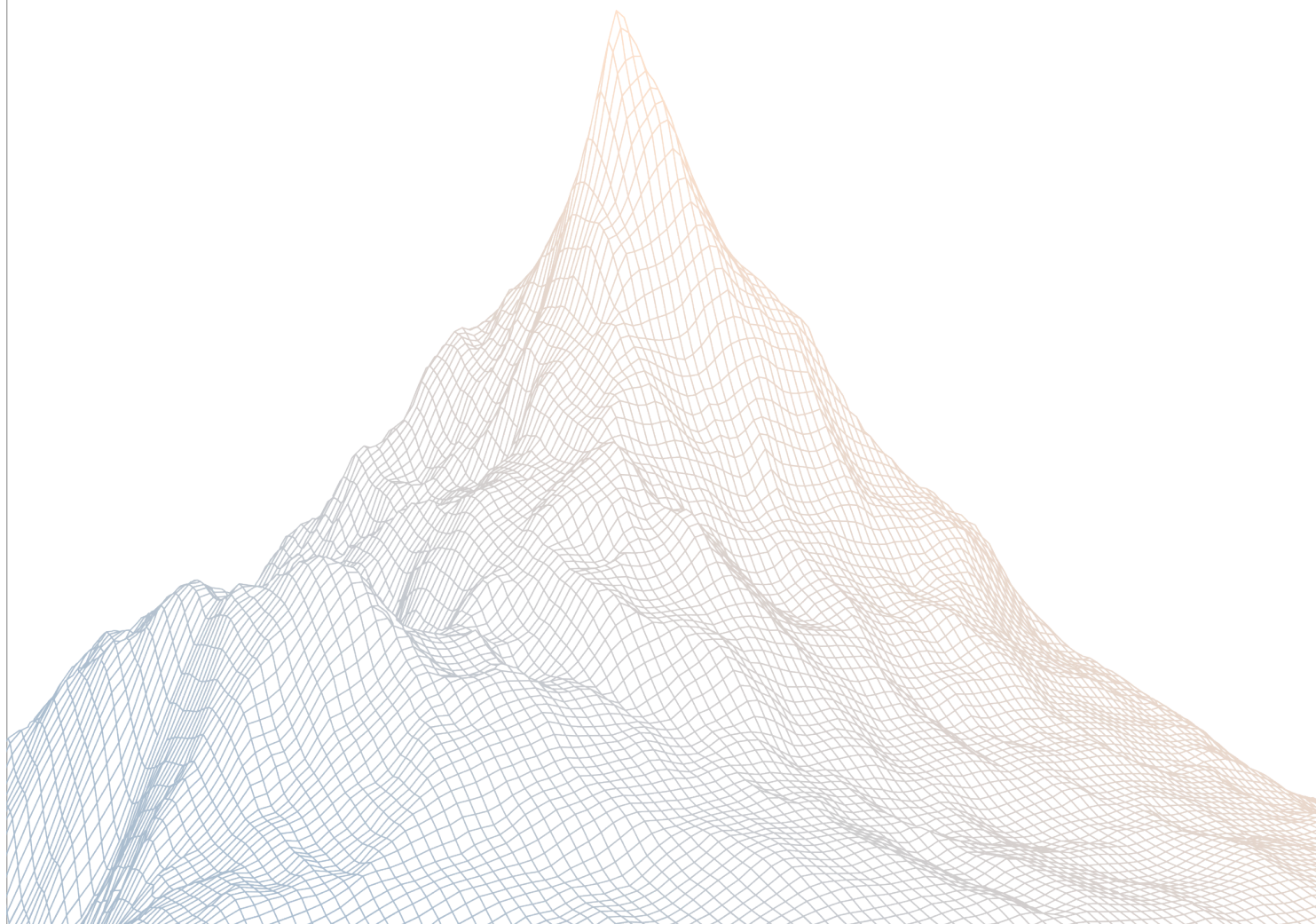


GMX Solana

Smart Contract Security Assessment

VERSION 1.1



Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About GMX Solana Protocol	4
2.2	Scope	4
2.3	Audit Timeline	5
2.4	Issues Found	5
3	Findings Summary	5
4	Findings	6
4.1	Medium Risk	7
4.2	Low Risk	9
4.3	Informational	11

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

2.1 About GMX Solana Protocol

Executive Summary

GMX Solana is a decentralized spot and perpetual exchange that supports low swap fees and low price impact trades. Trading is supported by unique multi-asset pools that earns liquidity providers fees from market making, swap fees and leverage trading. Dynamic pricing is supported by Chainlink Oracles and an aggregate of prices from leading volume exchanges.

2.2 Scope

The engagement involved a review of the following targets:

Target	gmx-solana
---------------	------------

Repository	https://github.com/gmsol-labs/gmx-solana/
-------------------	---

Commit Hash	3967d2e1e15852e659c225369f62a60fc5749090
--------------------	--

Files	Diff from 6b9ddef72a60577a7f832cd84e4baad50c113ae2 crates/chainlink-datastreams/src/report.rs
--------------	--

Target	Mitigation Review
---------------	-------------------

Repository	https://github.com/gmsol-labs/gmx-solana/
-------------------	---

Commit Hash	1156514934791a22fb5576799f3b8912de67a12e
--------------------	--

Files	Diff from 6b9ddef72a60577a7f832cd84e4baad50c113ae2 crates/chainlink-datastreams/src/report.rs
--------------	--

2.3 Audit Timeline

May 1, 2026	Audit start
May 4, 2026	Audit end
May 18, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	1
Informational	1
Total Issues	3

3

Findings Summary

ID	Description	Status
M-1	v11 bid and ask are used as fresh oracle min/max prices even though lastSeenTimestampNs only applies to mid	Acknowledged
L-1	marketStatus Unknown is rejected instead of persisting a non-open status	Acknowledged
I-1	decode_full_report can panic on malformed ABI offsets	Resolved

4

Findings

4.1 Medium Risk

A total of 1 medium risk findings were identified.

[M-1] v11 bid and ask are used as fresh oracle min/max prices even though lastSeenTimestampNs only applies to mid

SEVERITY: Medium

IMPACT: High

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [report.rs#L292](#)
- [gmsol.rs#L85](#)

Description:

decode parses v11 Chainlink reports and from_chainlink_report converts them into the on-chain PriceFeedPrice used by GMX oracle pricing.

In the V11 doc, Chainlink explicitly states the following regarding lastSeenTimestampNs:

IMPORTANT: The lastSeenTimestampNs field reflects the timestamp of the last update for the mid price only. Do not assume this timestamp applies to bid, ask, bidVolume, askVolume, or lastTradedPrice. Different fields may have been updated at different times.

<https://docs.chain.link/data-streams/reference/report-schema-v11#last-seen-timestamp>

However, the code uses lastSeenTimestampNs to perform staleness check on mid while also storing bid and ask as the oracle min_price and max_price.

In the event stale or session-transition bid and ask values are used, they affect the min/max prices for swaps, position execution, liquidations, PnL, and fee updates.

Example Scenario:

1. A v11 report has fresh mid = 100e18 with a non-stale lastSeenTimestampNs.

2. The same report carries stale session-transition quotes `bid = 80e18` and `ask = 120e18`.
3. `from_chainlink_report` accepts the report because `bid ≤ mid ≤ ask`.
4. `PriceFeedPrice::new` stores `80e18` as `min_price` and `120e18` as `max_price`.
5. Execution paths use those stale bounds through `pick_price(false)` and `pick_price(true)`, causing unfair execution or incorrect liquidation outcomes.

In `decode`:

```
let bid = bigint_to_signed(report.bid)?;  
let ask = bigint_to_signed(report.ask)?;  
last_update_timestamp: Some(report.last_seen_timestamp_ns),
```

In `from_chainlink_report`:

```
let mut price = Self::new(  
    Report::DECIMALS - divisor_decimals,  
    i64::from(observations_timestamp),  
    (price / divisor).try_into().unwrap(),  
    (bid / divisor).try_into().unwrap(),  
    (ask / divisor).try_into().unwrap(),  
    last_update_diff_secs.unwrap_or(0),  
);
```

Recommendations:

For v11, use `mid` as `min = max = mid`, or gate `bid` and `ask` with max-spread checks as suggested in [Chainlink's 24/5 US Equities guide](#).

GMX Solana: Acknowledged. Will consider using `max_deviation_factor` to mitigate this.

4.2 Low Risk

A total of 1 low risk findings were identified.

[L-1] `marketStatus Unknown` is rejected instead of persisting a non-open status

SEVERITY: Low

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [gmsol.rs#L36](#)
- [feed.rs#L167](#)

Description:

`from_chainlink_report` converts Chainlink market status into `PriceFeedPrice`, and `PriceFeed::check_and_get_price` decides whether stored prices can be consumed by execution paths.

Chainlink's 24/5 US Equities guide states:

Consumers must treat `marketStatus = 0` (Unknown) as a valid state and define deterministic behaviors when it is returned.

<https://docs.chain.link/data-streams/rwa-streams/24-5-us-equities-user-guide#market-status-unknownunavailable-marketstatus—0>

However, `v11 marketStatus = 0` is rejected as `UnknownMarketStatus`, so the price feed account is not updated to an explicit unknown or non-tradable state.

If the previous report marked the market open and remains within heartbeat, normal execution can continue temporarily even after the newest verified report says the market status cannot be determined.

Example Scenario:

1. At timestamp `1000`, the feed stores an open `v11` report with price `100e18`.

2. At timestamp 1010, Chainlink publishes a verified v11 report with `marketStatus = 0`.
3. `from_chainlink_report` rejects the report as `UnknownMarketStatus`.
4. The on-chain `PriceFeed` remains on the previous open report until heartbeat expiry.
5. Orders can continue using the prior open price even though the newest verified report indicated unknown market status.

In `from_chainlink_report`:

```
let mut is_open = match report.market_status() {  
  MarketStatus::Unknown => {  
    return Err(crate::Error::UnknownMarketStatus);  
  }  
  MarketStatus::Closed => false,  
  MarketStatus::Open => true,  
};
```

In `PriceFeed::check_and_get_price`:

```
if !allow_closed {  
  require!(is_open, CoreError::MarketNotOpen);  
}
```

Recommendations:

Accept unknown-status reports and persist an `Unknown` status in the price feed account so that trading is paused.

Alternatively, define bounded fallback behavior for `Unknown`, such as allowing restricted trading within a bounded range or referencing a tokenized asset price as suggested by [Chainlink](#).

GMX Solana: Acknowledged. A planned improvement of the market closed state management is going to mitigate this.

4.3 Informational

A total of 1 informational findings were identified.

[\[I-1\] decode_full_report can panic on malformed ABI offsets](#)

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [report.rs#L413](#)

Description:

`decode_full_report()` decodes the ABI-encoded full report context and report blob from caller-provided bytes.

However, it validates only that the blob offset is inside the payload before slicing `payload[offset..offset + 32]`, and later adds `offset + Report::WORD_SIZE + length` without checked arithmetic.

Malformed full reports can therefore panic during decoding instead of returning `Err`.

```
if offset < 128 || offset ≥ payload.len() {
    return Err(ReportError::InvalidLength("offset"));
}

let length = usize::from_be_bytes(
    payload[offset..offset + 32][24..Report::WORD_SIZE]
        .try_into()
        .map_err(|_| ReportError::ParseError("length as usize"))?,
);

if offset + Report::WORD_SIZE + length > payload.len() {
    return Err(ReportError::InvalidLength("bytes data"));
}
```

Recommendations:

Validate all ABI ranges with checked arithmetic before slicing, and return `ReportError::InvalidLength` whenever the offset, length word, or final blob range is outside the payload.

GMX Solana: Resolved with [@1156514934 ...](#)

Zenith: Verified.